

WEEK-11

TASK-1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#define NUM_THREADS 4

#define ITERATIONS 1000000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Initial counter value: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }
```

```

}

for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_join(threads[i], NULL) != 0)
    {
        perror("pthread_join");
        return 1;
    }
}

printf("Expected counter value: %d\n",
      NUM_THREADS * ITERATIONS);

printf("Actual counter value: %d\n", counter);

if (counter == NUM_THREADS * ITERATIONS)
    printf("No race condition observed in this run.\n");
else
    printf("Race condition occurred!\n");

return 0;
}

```

```

root@LAPTOP-RJFDBEBT:~/inode_test# nano counter.c
root@LAPTOP-RJFDBEBT:~/inode_test# gcc counter.c -o counter -pthread
root@LAPTOP-RJFDBEBT:~/inode_test# ./counter
Initial counter value: 0
Expected counter value: 4000000
Actual counter value: 1957476
Race condition occurred!
root@LAPTOP-RJFDBEBT:~/inode_test# counter++;
counter++: command not found
root@LAPTOP-RJFDBEBT:~/inode_test# ls -l counter.c counter
-rwxr-xr-x 1 root root 16264 Sep 27 14:39 counter
-rw-r--r-- 1 root root 1151 Sep 27 14:38 counter.c
root@LAPTOP-RJFDBEBT:~/inode_test# cat counter.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Initial counter value: %d\n", counter);

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    /* Wait for all threads */
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
        {
            perror("pthread_join");
            return 1;
        }
    }

    printf("Expected counter value: %d\n",
          NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);

    if (counter == NUM_THREADS * ITERATIONS)
        printf("No race condition observed in this run.\n");
    else
        printf("Race condition occurred!\n");

    return 0;
}
root@LAPTOP-RJFDBEBT:~/inode_test# rm counter
root@LAPTOP-RJFDBEBT:~/inode_test# gcc counter.c -o counter -pthread
root@LAPTOP-RJFDBEBT:~/inode_test# ./counter
Initial counter value: 0
Expected counter value: 4000000
Actual counter value: 2478545
Race condition occurred!
root@LAPTOP-RJFDBEBT:~/inode_test#

```

TASK-2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#define NUM_THREADS 4

#define ITERATIONS 1000000

int counter = 0;

pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    printf("Initial counter value: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");

            return 1;
        }
    }
}
```

```

    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
        {
            perror("pthread_join");
            return 1;
        }
    }

    printf("Expected counter value: %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);

    if (counter == NUM_THREADS * ITERATIONS)
        printf("Synchronization successful!\n");
    else
        printf("Unexpected result!\n");

    pthread_mutex_destroy(&lock);

    return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~/inode_test# nano counter_mutex.c
root@LAPTOP-RJFD8EBT:~/inode_test# gcc counter_mutex.c -o counter_mutex -pthread
root@LAPTOP-RJFD8EBT:~/inode_test# ./counter_mutex
Initial counter value: 0
Expected counter value: 4000000
Actual counter value: 4000000
Synchronization successful!
root@LAPTOP-RJFD8EBT:~/inode_test# ./counter
Initial counter value: 0
Expected counter value: 4000000
Actual counter value: 1863424
Race condition occurred!
root@LAPTOP-RJFD8EBT:~/inode_test# ./counter_mutex
Initial counter value: 0
Expected counter value: 4000000
Actual counter value: 4000000
Synchronization successful!
root@LAPTOP-RJFD8EBT:~/inode_test# ./counter
Initial counter value: 0
Expected counter value: 4000000
Actual counter value: 1903099
Race condition occurred!
root@LAPTOP-RJFD8EBT:~/inode_test# |

```